

Table of contents

Introduction aux Métaheuristiques	2
Commençons	2
Exemple introductif	2
Un problème d'optimisation difficile	2
Espace de Recherche	3
Optimisation non contrainte	3
Optimisation contrainte	3
Définitions et Notations	3
Structure de l'Espace de Recherche	4
Exemples	4
Optimisation continue	4
Programmation linéaire	4
Problème du sac à dos (Knapsack)	5
Problème du voyageur de commerce (TSP)	5
Problèmes NK	5
Plus sur les problèmes NK	5
Révision de la Complexité Algorithmique	6
Complexité et algorithmes efficaces	6
Mesurer la complexité : la notation "grand O"	6
Complexité algorithmique et classes P	6
Intraçabilité des problèmes exponentiels	7
Classes de complexité	7
Classes de complexité et optimisation difficile	7
Diagramme des classes de complexité	8
Exemple : Cycle hamiltonien	8
Le TSP détermine si un cycle hamiltonien existe	8
Le TSP est un problème NP-hard	8
Classes de complexité : un résumé	8
Hard-optimization	9
Métaheuristiques	9
Définition et contexte	9
Heuristiques vs. Métaheuristiques	9
Caractérisation des métaheuristiques (1/2)	10
Caractérisation des métaheuristiques (2/2)	10
Principes de Fonctionnement	11
Représentation et trajectoire (1/2)	11
Représentation et trajectoire (2/2)	11
Comment définir le voisinage ?	11
Exemple #1 : Mouvements dans $S = \mathbb{Z}^2$	12
Exemple #2 : Permutations (1/2)	12

Exemple #2 : Permutations (2/2)	12
Notes sur le voisinage	13
Stratégies d'exploration élémentaires	13
Recherche locale itérée (Iterated Local Search)	14
Paysage d'énergie/fitness (Energy/Fitness Landscape)	14
Lissage de fitness (Fitness Smoothing)	14
Paysage de fitness et exploration	15
Exemple : MaxOne (1/2)	15
Exemple : MaxOne (2/2)	15
Méthodes de population (1/2)	16
Méthodes de population (2/2)	16

Introduction aux Métaheuristiques

Commençons

Exemple introductif

Considérons n personnes dans un espace 2D, chacune aux positions x_i , $i = 1, \dots, n$.

Problème : Trouver le point y où ces personnes devraient se rencontrer pour minimiser la somme des distances parcourues par chacune d'entre elles.

Qu'est-ce que ce point ? La **médiane géométrique**, par définition (voir exemple 1D) :

$$\operatorname{argmin}_{y \in \mathbb{R}^2} \sum_{i=1}^n \|x_i - y\|$$

Il n'existe pas de solution analytique, mais un algorithme itératif : l'**algorithme de Weiszfeld** :

$$y_{k+1} = \left(\sum_{i=1}^n \frac{x_i}{|x_i - y_k|} \right) / \left(\sum_{i=1}^n \frac{1}{|x_i - y_k|} \right)$$

Solution numérique par **hill climbing** (ou **steepest descent**).

Un problème d'optimisation difficile

Les problèmes d'optimisation réels peuvent présenter des paysages de fitness très complexes avec de nombreux optima locaux, rendant la recherche de l'optimum global particulièrement difficile.

Espace de Recherche

Optimisation non contrainte

Classiquement, on résout un problème d'optimisation spécifié comme suit :

Trouver $x \in S$ tel que $f(x) \in \mathbb{R}$ soit maximal/minimal.

$$x_{\text{opt}} = \operatorname{argmin}/\operatorname{argmax}_{x \in S} f(x)$$

$$f_{\text{opt}} = \min / \max_{x \in S} f(x)$$

Optimisation contrainte

Habituellement, dans les problèmes d'optimisation, x peut être soumis à des contraintes données (par exemple $\|x\| < b$).

Les contraintes peuvent être exprimées en restreignant S aux solutions acceptables.

Définitions et Notations

S est l'espace de recherche

- S peut être discret, infini ou continu
- Habituellement, ici S est très grand
- Les éléments x de S sont appelés **solutions possibles** ou **configurations**

La fonction f est appelée : fonction objectif, fonction de coût, fonction de perte, **fitness**, ou énergie : $f : S \rightarrow \mathbb{R}$

Solution optimale $x_{\text{opt}} : f(x_{\text{opt}}) \geq f(y), \forall y \in S$

Valeur optimale : $f_{\text{opt}} = f(x_{\text{opt}})$

Unicité de x_{opt} et f_{opt} ? Optimum global ou local

Structure de l'Espace de Recherche

En général, problèmes multidimensionnels : $x = (x_1, x_2, \dots, x_n)$.

n est la **taille du problème**, ou le **nombre de degrés de liberté**.

La **cardinalité de S croît exponentiellement** avec la taille du problème :

$$x_i \in S_i \quad S = S_1 \times S_2 \dots \times S_n \quad |S| = |S_i|^n$$

La difficulté est que S est trop grand pour une exploration exhaustive et systématique.

Les métaheuristiques offrent un moyen d'explorer S quand aucun algorithme polynomial exact n'existe.

Les métaheuristiques contrastent avec l'**optimisation convexe** pour laquelle de nombreux algorithmes existent.

Exemples

Optimisation continue

$S = \mathbb{R}^n$: Problème d'optimisation continue. Dérivées (numériques), multiplicateurs de Lagrange, optima locaux/globaux. Ceux-ci doivent être examinés individuellement et prendre en compte les effets de bord.

Multiplicateur Lagrangien

Maximiser $f(x_1, x_2)$ avec la contrainte que $g(x_1, x_2) = 0$

$$\nabla f = \lambda \nabla g$$

où λ est un multiplicateur de Lagrange.

Programmation linéaire

Programmation linéaire : méthode pour atteindre le meilleur résultat (tel que le profit maximal ou le coût le plus bas) dans un modèle mathématique dont les exigences et l'objectif sont représentés par des relations linéaires.

Maximiser $z = f(x) = \sum_i c_i x_i$ avec les contraintes suivantes $\sum_i a_{ij} x_i \leq b_j$, $x \in S = \mathbb{R}^n$.

Les solutions sont à la frontière d'un polygone convexe (algorithme **SIMPLEX**).

Les choses deviennent compliquées si $x \in S = \{0, 1\}^n$.

Problème du sac à dos (Knapsack)

n objets de valeur p_i avec taille w_i . Maximiser $\sum p_i x_i$ avec $\sum w_i x_i \leq c$, $x_i \in \{0, 1\}$, $c > 0$ taille du sac.

Problème du voyageur de commerce (TSP)

Problème d'optimisation combinatoire (i.e., **quadratic assignment problem** - QAP).

Problèmes NK

Problème MaxOne

Problèmes NK (S. Kauffman, réseau de régulation génétique). N gènes, liés à K autres : graphe avec N nœuds et degré K .

$$x \in S = \{0, 1\}^N$$

$$f(x) = \sum_i^N f_i(x_i, \text{neighbors}_i)$$

avec $K - 1$ voisins de x_i . **MaxOne** : $K = 1$.

Cas $K = 3$: $h(111)$ ou $h(110)$ est max.

Les problèmes NK sont théoriquement intéressants car ils offrent des espaces de recherche synthétiques, en particulier lorsque N et K augmentent.

Plus sur les problèmes NK

Considérons N personnes avec deux stratégies commerciales notées par $x_i = 0$ ou $x_i = 1$ (type binaire).

Le succès de chaque stratégie dépend de manière critique des stratégies des autres personnes (interdépendance - systèmes complexes), et de leur relation (coopération vs. compétition).

Chaque individu est influencé par K autres. La fonction indique le profit/gain résultant associé à la stratégie adoptée par l'agent i compte tenu de l'interdépendance avec le reste du collectif.

Le profit/gain collectif global pour le système de N agents est simplement la somme $f = \sum f_i$.

Révision de la Complexité Algorithmique

Complexité et algorithmes efficaces

Pour trouver une solution optimale, on a besoin d'un algorithme efficace (évidemment !).

L'effort de calcul requis par un algorithme est donné par la quantité de mémoire requise M et le temps de calcul T .

M et T sont respectivement la **complexité en espace** et la **complexité en temps** de l'algorithme.

Habituellement, on considère la complexité du pire cas, ou celle du cas moyen.

On considère généralement la **complexité asymptotique**, c'est-à-dire la relation $M(n)$ et $T(n)$ en fonction de la taille du problème n .

Mesurer la complexité : la notation "grand O"

La complexité est exprimée avec une expression comme " $O()$ ".

Si on dit que $T(n) = O(f(n))$, cela signifie qu'il existe une constante k et une valeur n_0 telles que si $n \geq n_0$ alors $T(n) \leq kf(n)$.

Par exemple :

- $T(n) = 0.5n^2 + 2n = O(n^2)$
- $T(n) = n + \log(n) < n + n = O(n)$

Complexité algorithmique et classes P

Pour certains problèmes (tri, multiplication de matrices, etc.), un algorithme est connu qui les résout en temps polynomial :

$$T(n) = O(n^m)$$

avec m généralement petit. Ces problèmes appartiennent à la **classe P**.

Pour d'autres problèmes, aucun algorithme polynomial n'est connu (ou n'existe !). Il n'y a que des algorithmes exponentiels. Par exemple :

$$T(n) = O(2^n)$$

Ces problèmes sont **difficiles sur le plan computationnel** car $T(n)$ devient excessivement grand lorsque n augmente même modérément.

Intraçabilité des problèmes exponentiels

n	10	50	100	1000
$\log_2 n$	3	6	7	10
n^3	1000	125'000	10^6	10^9
2^n	1024	1.3×10^{15}	1.27×10^{30}	1.05×10^{301}
$n!$	3629×10^3	3.041×10^{64}	10^{158}	4×10^{2567}

Classes de complexité

En théorie de la complexité, il existe des problèmes dits appartenir aux classes **P**, **NP**, **NP-complet** ou **NP-difficile** (NP-hard).

La classe **P** contient les problèmes qui peuvent être résolus par un algorithme polynomial.

NP signifie polynomial non déterministe (non-deterministic polynomial) car ils peuvent être définis par une machine de Turing non déterministe.

Les problèmes **NP** sont ces problèmes pour lesquels une solution donnée peut être vérifiée en temps polynomial.

Ainsi, les problèmes **P** sont évidemment dans **NP**.

Classes de complexité et optimisation difficile

Les problèmes **NP-hard** sont ceux dont la solution résout n'importe quel problème **NP**, à une transformation polynomiale près.

Les problèmes qui sont à la fois dans les classes **NP-hard** et **NP** forment la classe **NP-complet**.

Les problèmes **NP-complets** comprennent :

- La coloration de graphes
- Les problèmes de satisfaction
- La recherche de cycles hamiltoniens dans un graphe

Les problèmes **NP-complets** et **NP-hard** sont résolus avec des algorithmes exponentiels.

Diagramme des classes de complexité



Exemple : Cycle hamiltonien

Trouver un chemin fermé qui passe par tous les nœuds une fois et une seule.

Note : La longueur du chemin est 4 et est égale au nombre de nœuds.

Le TSP détermine si un cycle hamiltonien existe

TSP (Traveling Salesman Problem) : trouver le chemin le plus court qui passe par tous les nœuds une fois et une seule, en supposant un graphe complètement connecté. Ajouter les arêtes manquantes avec un poids 2.

- Pas de cycle hamiltonien → Le tour le plus court est de longueur 5 et non 4

Le TSP est un problème NP-hard

- Le TSP n'est **pas dans NP** car on ne peut pas vérifier qu'un tour est de longueur minimale en temps polynomial
- Trouver un cycle hamiltonien n'est **pas dans P** car aucun algorithme n'est connu pour trouver un cycle hamiltonien en temps polynomial
- Trouver un cycle hamiltonien est connu pour être **NP-complet** (difficile à démontrer)
- Le TSP résout le problème du cycle hamiltonien
- Le TSP est donc **NP-hard**

Classes de complexité : un résumé

Classe P : Tri (sorting)

Classe NP : Factorisation de grands entiers (Factoring large integers)

Classe NP-complet : Cycle hamiltonien (Hamiltonian cycle)

Classe NP-Hard : TSP

Hard-optimization

De nombreux problèmes d'optimisation qui nécessitent des métaheuristiques sont **NP-hard**. On les appelle des problèmes de **Hard-optimization**.

Une solution déterministe nécessiterait un algorithme exponentiel.

Les métaheuristiques sont alors utilisées.

Mais, quelle est alors la complexité d'une métaheuristique ?

Métaheuristiques

Définition et contexte

Métaheuristiques : terme inventé par Fred Glover en 1986 dans le contexte de la recherche **tabu** (voir Chap. 2).

Pour de nombreux problèmes réels (Recherche Opérationnelle - OR), aucun algorithme polynomial n'existe.

→ Mais pour d'autres problèmes oui !

Si n est grand, l'espace de recherche S devient trop vaste pour utiliser un algorithme exponentiel.

Métaheuristiques : destinées à explorer S de manière quelque peu intelligente pour trouver des optimums.

→ Espérons de manière plus efficace qu'à travers un échantillonnage purement aléatoire.

Compromis entre qualité de la solution et coût computationnel (i.e. temps CPU).

→ Capable de retourner une solution acceptable dans un temps raisonnable.

Heuristiques vs. Métaheuristiques

Heuristiques : méthodes d'exploration qui exploitent certains aspects spécifiques du problème en question et ne s'appliquent qu'à lui.

→ Par exemple, lors de la résolution d'un problème de programmation linéaire par l'algorithme du simplexe, une heuristique est souvent utilisée pour choisir les variables entrantes et sortantes.

Métaheuristiques : méthodes d'exploration générales, presque toujours stochastiques, qui s'appliquent de la même manière à de nombreux problèmes différents.

→ Les exemples sont la recherche **tabu**, le **simulated annealing**, les **ant colony**, et les algorithmes évolutionnaires. Sujets de ce cours !!

Algorithmes d'approximation : cherchent à trouver des solutions à des problèmes spécifiques avec une qualité choisie et avec des limites spécifiques en termes de temps CPU.

Caractérisation des métaheuristiques (1/2)

Aucune hypothèse sur les propriétés mathématiques de f .

→ La seule exigence est que f puisse être calculée $\forall x \in S$.

Basées sur un nombre de **paramètres** qui guident l'exploration, et qui affectent la qualité de la recherche.

→ Leurs valeurs optimales sont souvent trouvées empiriquement pour un problème donné.

Nécessitent un **point de départ** (configuration initiale).

→ Soit sélectionné aléatoirement, soit basé sur d'autres stratégies.

Nécessitent une **condition d'arrêt** : par exemple, limite CPU, borne sur la qualité de la solution trouvée, etc.

Caractérisation des métaheuristiques (2/2)

En général, elles ont une **composante stochastique** et peuvent être inspirées par des processus naturels.

Souvent, elles ont des exigences CPU importantes (pas bon marché !).

Elles sont **faciles à implémenter** et peuvent facilement être parallélisées.

Classiquement, elles combinent deux actions clés :

intensification et **diversification**, plus communément **exploitation** et **exploration** : soit on cherche des solutions plus prometteuses dans S , soit on analyse davantage une région d'intérêt trouvée.

Règle d'or : ne jamais utiliser de métaheuristiques si une autre méthode peut être appliquée !

Principes de Fonctionnement

Représentation et trajectoire (1/2)

Il y a un **besoin critique** de trouver une représentation appropriée du problème dans l'espace de recherche donné S :

→ i.e., comment coder les solutions possibles ?

Une métaheuristique est une **trajectoire** dans S .

Pour définir cette trajectoire et le mouvement le long de celle-ci, les ingrédients suivants sont requis :

(1) **Voisinage** $V(x)$ pour tout $x \in S$: ce sont les y atteignables depuis x .

Représentation et trajectoire (2/2)

(2) **Opérateur de recherche/exploration** U qui sélectionne parmi les voisins du point actuel le long de la trajectoire d'exploration :

$$x_0 \xrightarrow{U} x_1 \in V(x_0) \xrightarrow{U} x_2 \in V(x_1) \dots$$

Ce processus continue jusqu'à ce que le critère d'arrêt soit activé.

Selon la métaheuristique, la sortie est soit :

- (i) le point x correspondant à la fitness optimale $f(x)$ trouvée le long de la trajectoire
- (ii) ou le dernier point x le long de la trajectoire de recherche

Comment définir le voisinage ?

Trouver des moyens d'exprimer explicitement $V(x)$ pour tout $x \in S$:

→ Souvent impraticable, voire impossible.

En général, le voisinage est spécifié au moyen de ℓ **transformations élémentaires** T_i (ou mouvements) telles que :

$$T_i(x) \in V(x)$$

$$V(x) = \{x' \in S \mid x' = T_i(x), i = 1, \dots, \ell\}$$

Exemple #1 : Mouvements dans $S = \mathbb{Z}^2$

$$x \in S \Rightarrow x = (i, j)$$

Par exemple, on peut considérer quatre mouvements cardinaux ($\ell = 4$) notés **NORTH**, **EAST**, **SOUTH** et **WEST** tels que :

- $N(i, j) = (i, j + 1)$
- $E(i, j) = (i + 1, j)$
- $S(i, j) = (i, j - 1)$
- $W(i, j) = (i - 1, j)$

Exemple #2 : Permutations (1/2)

Considérons l'espace de recherche S des permutations de n objets (cardinal = $n!$).

Avec $n = 3$, nous avons :

$$A = \{(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)\}$$

avec $|A| = 3! = 6$.

On peut définir un voisinage d'une permutation donnée par l'ensemble des permutations qui peuvent être générées par une opération donnée.

Par exemple, la transposition (i, j) , qui consiste à échanger les positions i et j .

Exemple #2 : Permutations (2/2)

Pour $n = 5$, les mouvements sélectionnés sont :

$$m_i \in \{(1, 2), (1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5), (3, 4), (3, 5), (4, 5)\}$$

Par exemple, la permutation $(a_3, a_2, a_4, a_5, a_1)$ admet $(a_2, a_3, a_4, a_5, a_1)$ comme voisin lors de l'application du mouvement $(1, 2)$ qui transpose les deux premiers éléments.

→ Toutes les permutations sont un produit fini de transpositions.

Notes sur le voisinage

Certaines métaheuristiques **n'explorent pas le voisinage complet** avant de passer au point suivant.

→ Recherche non exhaustive.

Typiquement, elles utilisent un élément stochastique pour établir aléatoirement le point suivant, “en espérant” qu’il donnerait une valeur optimale pour f dans S .

Pour les métaheuristiques effectuant une recherche de voisinage exhaustive, nous pouvons définir une **structure hiérarchique du voisinage** :

- Si rien d’utile n’est trouvé au 1er niveau, on passe au 2ème niveau, et ainsi de suite
- Si rien de meilleur n’est trouvé, on arrête la recherche de voisinage et on passe à un autre voisinage

Recherche globale vs. locale

Stratégies d’exploration élémentaires

Marches aléatoires (Random walks)

Marches aléatoires dépendantes du voisinage

Hill climbing (très gourmand - déterministe ou stochastique)

Hill climbing probabiliste : probabilité de sélectionner un voisin $p(x) = f(x) / \sum f(x')$

...

Cas de l’algorithme de **backtracking** et **branch-and-bound** : ce ne sont PAS des métaheuristiques (exploration systématique de l’espace de recherche) mais ils partagent un certain nombre de caractéristiques avec les métaheuristiques :

- (i) Recherche et sélection de voisinage (enfants)
- (ii) Aucune propriété spécifique requise pour la fonction de fitness

Recherche locale itérée (Iterated Local Search)

Considérons une métaheuristique **localSearch** qui se bloque sur des optima locaux. On peut construire une métaheuristique plus puissante :

Réduire S à S' qui ne contient que les optima locaux de S .

Topologie induite sur S' : deux points sont voisins si leur bassin d'attraction reste comme tel pour S .

En pratique, nous cherchons à trouver les voisins de $x' \in S'$ en perturbant x' de manière aléatoire et en appliquant à nouveau la stratégie de recherche locale sur S pour trouver l'optimum local voisin dans S .

→ **Défi** : choisir la procédure de perturbation.

Paysage d'énergie/fitness (Energy/Fitness Landscape)

Représentation graphique de la fitness compte tenu de la topologie induite par le voisinage.

Nécessite une topologie de voisinage.

Un paysage de fitness NK (NK-Landscape, $N = 10$, $K = 3$, f_i construit aléatoirement)

Autre représentation : hypercube binaire avec 2^{10} points.

Lissage de fitness (Fitness Smoothing)

Fitness moyenne :

$$\bar{f} = \frac{1}{|S|} \sum_{x \in S} f(x)$$

Une fitness f_λ peut être lissée avec un coefficient λ :

$$f_\lambda(x) = \bar{f} + \lambda(f(x) - \bar{f})$$

Paysage de fitness NK lissé avec smoothing = 0.4 et smoothing = 0.8

Paysage de fitness et exploration

Paysage facile vs. difficile : sélection de la stratégie de codage, sélection du voisinage.

Cas difficile, mais facile : maximiser $f(x) = x_1 \cdot x_2 \dots x_n$ pour $x_i \in \{0, 1\}$. Le paysage est plat de sorte que rien ne peut vraiment guider la recherche.

Exemple de **MaxOne** avec deux stratégies de codage différentes.

Exemple : MaxOne (1/2)

Le problème **MaxOne** illustre l'importance du choix de l'espace de recherche et du voisinage.

MaxOne cherche à maximiser :

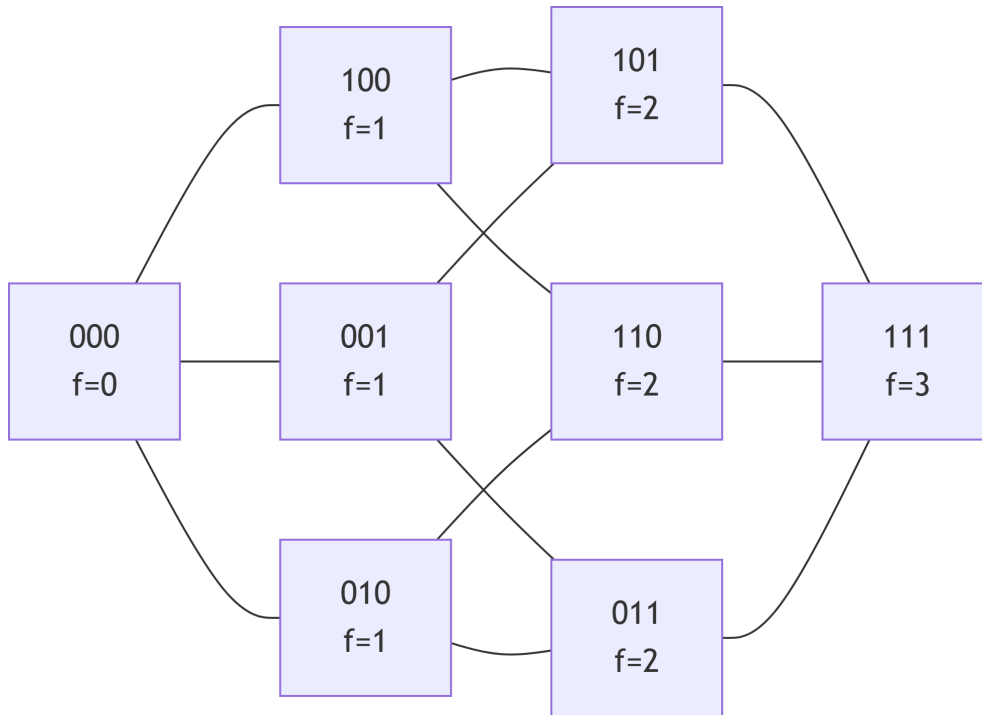
$$f(x_1, \dots, x_n) = \sum_{i=1}^n x_i$$

Solution évidente (pour nous !) : $x = (x_1, \dots, x_n) = (1, 1, \dots, 1)$

Espace de recherche : ensemble d'entiers de 0 à $2^n - 1$ (cardinal = 2^n)

Exemple : MaxOne (2/2)

MaxOne avec deux stratégies de codage distinctes : on interprète $(x_1, \dots, x_n)$ comme un entier. Le voisinage comprend $x - 1$ et $x + 1$. Si U choisit un voisin sans jamais diminuer la fitness, on finit par être bloqué. Cependant avec l'hypercube, ça marche ! (Cas $n = 3$ montré ci-dessous)



Méthodes de population (1/2)

Métaheuristiques à point unique vs. basées sur une population

A : Trois étapes d'une métaheuristique à point unique

- Étape 0 : x_0 dans S
- Étape 1 : x_1 dans S
- Étape 2 : x_2 dans S

B : Trois étapes d'une métaheuristique basée sur une population

- Étape 0 : $\{x_0, y_0, z_0\}$ dans S
- Étape 1 : $\{x_1, y_1, z_1\}$ dans S
- Étape 2 : $\{x_2, y_2, z_2\}$ dans S

Méthodes de population (2/2)

De nombreuses métaheuristiques considèrent une **population de solutions tentatives**. Ensuite, elles laissent cette population évoluer pour explorer S : par exemple, algorithmes génétiques/évolutionnaires.

La solution optimale est alors la meilleure solution trouvée à travers toutes les générations/évolutions.

Nous travaillons dans un **espace de recherche étendu** : S^N où N est le nombre d'individus dans la population.

Le voisinage d'une population est une autre population : les transformations élémentaires sont plus complexes puisqu'elles peuvent utiliser la population entière, et pas seulement une seule solution.

Cela peut être vu comme une sorte de **co-évolution** de nombreuses solutions.

Fin du Chapitre 1